

Memorial da disciplina de Redes Neurais e Algoritmos Genéticos

Lucas Nascimento da Silva

Ilum Escola de Ciência

Prof. Responsável: Dr. Daniel Roberto Cassar

Agradecimentos

Agradeço à contribuição inestimável de meus colegas Rômulo Emanuel Rabelo Cruz e José Victor da Silva Izidorio pela ajuda em alguns momentos da criação dessas atividades.

Contracapa

Oh, well, the night is long, the beads of time pass slow

Tired eyes on the sunrise

Waiting for the eastern glow

— The Battle of Evermore, Led Zeppelin

1 Atividade 3.1

Nome: “Eu podia jurar que não veria grafos nunca mais na minha vida” — aluno da turma 2024 que não quis se identificar

Objetivo: Resolver o grafo computacional da expressão abaixo em uma folha sulfite.

$$L = (y - (a \cdot b + c \cdot d + e \cdot f + g))^2$$

Foi dado que $y = -50$, $a = -2$, b é o último dígito da sua matrícula da Ilum, $c = 8$, $d = -5$, e é o último dia do seu aniversário, f é o seu primo favorito menor que 10 e $g = -10$. No meu caso o valor de b foi 5, de e foi 1 e f igual a 7.

Resolução: Utilizei da forma de representar grafos computacionais ensinados na primeira aula para desenhar meu grafo, respeitando coisas como representar letras a partir de retângulos e símbolos a partir de círculos. Como foi pedido na tarefa realizei o backpropagation me atentando a mostrar qual é a derivada parcial e regra da cadeia para cada vértice.

Conclusão e aprendizado: A tarefa tinha o objetivo de de testar o aprendizado dos alunos em relação à representação dos grafos computacionais.

Onde está a tarefa: Esse monstrinho está na pasta Monstrinhos/Monstrinho_3.1 no [GitHub](#) com o título Monstrinho_3.1.jpg.

2 Atividade 3.2

Nome: “Átomos não são bolinhas e ligações químicas não são pauzinhos” — Prof. Julio”

Objetivo: Utilize classes de Python para modelar elementos químicos e moléculas.

Considerações do experimento: Crie uma classe chamada Elemento. Todo elemento químico utilizado nesta tarefa deve ser uma instância desta classe. Toda instância da classe Elemento deve ter os seguintes atributos: símbolo do elemento, número atômico do elemento e peso atômico do elemento (fique à vontade para incluir outras informações se quiser). Crie uma classe chamada Molécula. Esta classe deve receber um dicionário onde as chaves representam os elementos e os valores representam a quantidade do respectivo elemento. A classe Molécula deve ter um método capaz de exibir o peso atômico da molécula criada e um método capaz gerar uma fórmula química (em string) para esta molécula. Observação: não é necessário que a fórmula química siga as regras oficiais da química, basta representar os elementos existentes e suas quantidades.

Resolução: Utilizei duas classes, uma chamada "Elemento", que guarda quais são as informações sobre o símbolo, o número atômico e o peso atômico de um elemento e outra classe chamada "Molécula" que recebe um dicionário com elementos definidos a partir da classe Elemento e a partir disso retorna a fórmula dessa molécula. Me atentei a fazer com que quando houvesse apenas 1 átomo de um elemento em uma molécula o número 1 não aparecesse, então, o que seria H2O1, vira H2O. [2]

Conclusão e aprendizado: As classes foram muito úteis nesse caso, elas foram capazes de fazer o cálculo da massa da molécula perfeitamente, além de apresentar a fórmula bem. Isso demonstra que as classes podem ser muito úteis em diversos casos.

Onde está a tarefa: Esse monstinho está na pasta Monstrinhos/Monstrinho_3.2 no [GitHub](#) com o título Monstrinho_3.2.ipynb.

3 Atividade 3.4

Nome: `__dunder__`

Objetivo: Se informe sobre métodos dunder que não foram utilizados no material de aula e crie uma classe que contenha pelo menos 3 destes métodos dunder. Faça códigos onde cada um destes métodos dunder seja acessado sem os chamar explicitamente (exemplo: não é para rodar `a.__add__(b)` mas sim `a + b` para o caso do dunder `__add__`).

Considerações do experimento: A classe deve conter pelo menos 3 métodos dunder que não foram vistos no material da disciplina. Sua classe deve fazer sentido, isto é, não crie uma classe “sem pé nem cabeça” apenas para a entrega. Reflita sobre uma classe onde os métodos dunder propostos realmente fazem sentido. Na sua entrega, explique brevemente o que fazem os métodos dunder que escolheu e mostre eles em ação com uma instância da sua classe.

Resolução: A ideia principal foi criar uma classe chamada “Pontuacao” que calcula quantos pontos um determinado jogador, grupo, time, etc, possui. A classe escolhida vai utilizar os métodos dunder `__iadd__` para adicionar pontos a um jogador, `__isub__` para subtrair pontos de um jogador, `__eq__` para conferir se dois jogadores estão com a mesma pontuação, `__lt__` e `__gt__` para conferir qual jogador possui mais pontos. Além disso defini uma função chamada “partida” que usa a classe Pontuacao para definir o resultado de um jogo de futebol contra a seleção brasileira, usando dessa função fiz a trajetória do Brasil na copa de 2002, ano no qual o Brasil conquistou seu pentacampeonato mundial. No final do notebook está uma parte extra na qual usei a biblioteca pygame para reproduzir o icônico áudio do Galvão Bueno narrando o pentacampeonato brasileiro. [3,4,5,6]

Conclusão e aprendizado: Os métodos dunder tiveram grande utilidade nesse caso, mesmo ele sendo simples. Mesmo usando uma classe simples é possível perceber o quão funcional pode ser utilizar os métodos dunder para realizar diversas tarefas em python. Algo que poderia melhorar, por exemplo, é a quantidade de atributos, ao invés de calcular `+=` em relação aos pontos é possível, por exemplo, adicionar em 1 a quantidade de partidas jogadas por cada jogador.

Onde está a tarefa: Esse monstinho está na pasta Monstrinhos/Monstrinho_3.4 no [GitHub](#) com o título Monstrinho_3.4.ipynb.

4 Atividade 3.6

Nome: A curva de aprendizado

Objetivo: Implemente o registro da curva de aprendizado tanto dos dados de treino quanto de validação no código de redes neurais feito em Python puro nesta disciplina. Além de implementar, treine um modelo de rede neural em um conjunto de dados qualquer e mostre e interprete o gráfico das curvas de aprendizado.

Comentário: observe que o enunciado diz claramente que é para realizar a tarefa na rede neural feita em Python puro nos vídeos da disciplina. Se você está usando o PyTorch, numpy, tensorflow, keras, lightning ou qualquer outra biblioteca pronta, você está no caminho errado!

Resolução: Para fazer esse monstrinho criei duas listas que registram a **loss** de treino e de validação e a partir delas plotei os gráficos usando o **matplotlib**. Utilizei o modelo de linguagem ChatGPT da OpenAI para gerar dados que tendiam a dar overfitting quando treinados.

Conclusão e aprendizado: Acrescentar o registro da curva de aprendizado é de grande utilidade para analisar como está a sua rede neural, se ela está diminuindo o erro corretamente. Observar comportamentos da curva em determinados momentos do treinamento pode ser crucial para mostrar se talvez pode ser interessante implementar uma estratégia para, por exemplo, evitar o overfitting da rede neural. Além disso, é conveniente utilizar esse gráfico para entender melhor o seu gráfico.

Onde está a tarefa: Esse monstrinho está na pasta Monstrinhos/Monstrinho_3.6 no [GitHub](#) com o título Monstrinho_3.6.ipynb.

5 Atividade 4.1

Nome: Quem classifica a classe classificadora?

Equipe: José Victor da Silva Izidorio e Lucas Nascimento da Silva.

Tarefa realizada por cada um dos integrantes: Ambos os integrantes do grupo estavam, ao mesmo momento, fazendo as alterações necessárias para completar o exercício, usando apenas um computador e comentando sobre o exercício.

Objetivo: Altere a modelo neural feita em Python puro para resolver um problema de classificação. Treine uma modelo neural em um dataset simples de classificação para mostrar que funciona.

Comentário: Aqui é necessário se informar sobre as diferenças de uma modelo neural classificadora com relação a uma modelo neural regressora. A função de perda, por exemplo, não poderá ser mais a função de perda dos resíduos quadrados.

Resolução: Para adaptar a modelo neural para um problema de classificação o ponto de partida inicial é adaptar a classe Valor para fazer duas novas funções.

São elas:

- Adicionar a função de ativação ReLU. Existe algumas vantagens em utilizar a função ReLU no lugar da sigmoid, como a maior simplicidade para calcular e evitar que os valores vão para os extremos ficando muito próximos a zero e dificultando o cálculo dos gradientes. [7]

- Calcular logaritmo para fazer o cálculo da função de perda CrossEntropy. [8]

Além de adaptar a classe Valor, foi necessário definir uma função para calcular a função de perda CrossEntropy.

- A segunda adaptação que fizemos foi apenas aplicar o OneHotEncoder nos dados de classificação para colocá-los na modelo neural.

Conclusão e aprendizado: A rede neural adaptada para resolver problemas de classificação mostrou-se muito útil, a capacidade de ter um modelo poderoso que consegue prever dados de classificação é de grande utilidade para diversas áreas, como a área da saúde com esse dataset que faz uma previsão em relação a uma doença cardíaca ou até mesmo em outras áreas da ciência.

Onde está a tarefa: Essa Fera está na pasta Feras_formidaveis/Fera_4.1 no [GitHub](#) com o título Fera_4.1.ipynb.

6 Atividade 4.2

Nome: Stop right now, thank you very much **Objetivo:** Implementar uma estratégia de Parada Antecipada no processo de treino da rede neural feita em Python puro.

Resolução: A estratégia Early Stopping consiste em parar o treinamento da rede neural assim que o seu desempenho para dados de validação começa a diminuir, evitando o overfitting.

A Figura 1) demonstra como funciona essa estratégia.

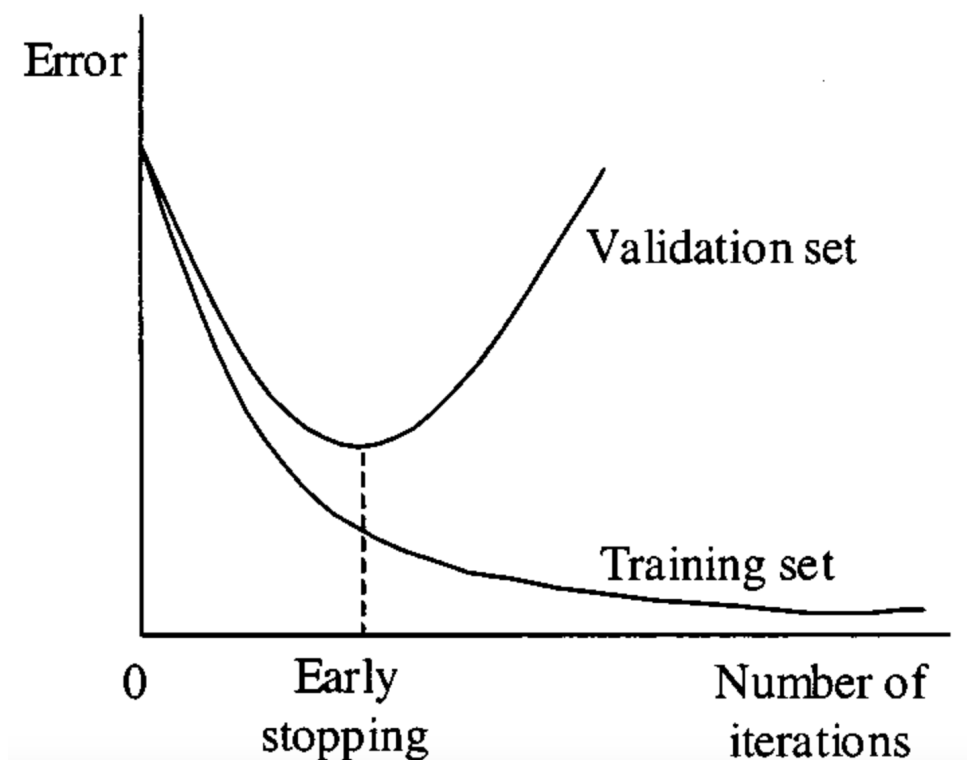


Figure 1: Funcionamento da estratégia de Early Stopping [10]

A ideia para implementar o Early Stopping (que nesse caso será feito em uma rede neural treinada com pytorch) é treinar a rede e a cada época avaliar o modelo com o conjunto de validação, se o modelo melhorar o treinamento continua e se não melhorar após um determinado número de épocas o código retorna o modelo que teve o menor erro para os dados de validação. [9,10,11]

Conclusão e aprendizado: A estratégia de Early Stopping se mostrou poderosa para o que propõe. Treinar o modelo com ela impede que a rede neural fique overfitada para os dados de treino ao mesmo tempo que melhora ao máximo a previsão para novos dados.

Onde está a tarefa: Essa Fera está na pasta Feras_formidaveis/Fera_4.2 no [GitHub](#) com o título Fera_4.2.ipynb.

7 Atividade 4.3

Nome: Derrube para fora

Objetivo: Implemente o regularizador dropout na rede neural feita em Python puro.

Resolução: Para a aplicação da estratégia de dropout na rede neural criada em python puro foram feitas alterações apenas nas classes Camada e MLP. Foi acrescentado um parâmetro para a classe MLP que controla a porcentagem de neurônios que serão desativados em uma época. A taxa de dropout passa para a classe Camada na qual o python confere se o modelo está em modo treino, se sim, uma máscara aleatória é feita a partir de uma lista que possui valores 1 ou 0, que são multiplicados pelos respectivos neurônios de modo a zerar alguns para o dropout ser aplicado. Além disso, os dados que foram retornados dos neurônios precisaram de um ajuste para que os neurônios que não foram zerados conseguissem suprir a falta dos neurônios zerados. [12,13,14]

Conclusão e aprendizado: O modelo com a estratégia de dropout funcionou bem, o erro para os dados de validação diminuiu, apesar dos neurônios zerados. Foi possível perceber a importância do cálculo para ajustar os neurônios não zerados para que eles pudessem suprir a falta dos demais. De uma maneira geral, a estratégia de dropout parece funcionar muito bem e pode ser muito útil em casos nos quais o overfitting é fácil de ser alcançado.

Onde está a tarefa: Essa Fera está na pasta Feras_formidaveis/Fera_4.3 no [GitHub](#) com o título Fera_4.3.ipynb.

8 Atividade 4.5

Nome: Um momento, por favor!

Objetivo: Implemente o otimizador de Descida do Gradiente com Momento (Gradient Descent with Momentum) na rede neural feita em Python puro..

Resolução: O otimizador de Descida do Gradiente com Momento tem o intuito de acelerar a convergência de uma rede neural por meio da atualização dos parâmetros na direção oposta ao gradiente de erro. Isso é feito a partir de uma variável velocidade que acumula as atualizações das épocas anteriores, logo, se a rede estiver indo em uma mesma direção essas velocidades serão aumentadas, tornando a convergência mais rápida.

A alteração feita no código da rede neural em python puro foi realizada apenas no loop de treinamento do modelo, a partir da criação do dicionário que armazena a velocidade para cada parâmetro da MLP antes do modelo começar a ser treinado. Dentro do loop, no momento em que os parâmetros são atualizados, eles são atualizados utilizando a fórmula para Descida do Gradiente com Momento atualizando as velocidades do dicionário velocidades. [15,16,17]

Conclusão e aprendizado: O modelo funcionou bem em aplicar a descida do gradiente com momento e a aplicação dele foi simples, mostrando a eficiência desse otimizador. Além disso, foi interessante perceber o quão diferente é o comportamento do erro durante o treinamento do modelo, que chega rapidamente a um mínimo e se mantém relativamente constante até chegar em outro mínimo, formando uma espécie de escada.

Onde está a tarefa: Essa Fera está na pasta Feras formidaveis/Fera_4.5 no [GitHub](#) com o título Fera_4.5.ipynb.

9 Atividade 4.6

Nome: E se meus dados forem imagens?

Objetivo: Implementar uma rede neural convolucional (CNN) utilizando PyTorch ou lightning. Treine esta rede neural em um conjunto de dados de imagens. Explique para o leitor como funciona a camada de convolução de uma CNN e o motivo de utilizarmos este tipo de arquitetura quando estudamos imagens.

Resolução: A grande vantagem de usar CNN para trabalhar com imagens se dá pelo fato de que em imagens, os pixels sozinhos não trazem tanta informação quanto o contexto geral dos pixels que formam um determinado local da imagem. Sendo assim, é possível observar padrões em locais específicos da imagem que podem trazer informações importantes para a previsão.

A CNN foi treinada a partir do módulo torch utilizando duas camadas convolucionais e uma camada de agrupamento. Todo o processo de funcionamento de uma rede neural convolucional está detalhado no arquivo notebook referente a essa atividade. [18]

Conclusão e aprendizado: O módulo torch é de grande utilidade para treinar redes neurais, especialmente as mais complexas como é o caso de uma Rede Neural Convolucional. Foi possível perceber a utilidade que existe em uma CNN e como ela pode ser usada para prever dados de imagens.

Onde está a tarefa: Essa Fera está na pasta Feras_formidaveis/Fera_4.6 no [GitHub](#) com o título Fera_4.6.ipynb.

10 Atividade 4.9

Nome: Senha de tamanho variável

Equipe: José Victor da Silva Izidorio e Lucas Nascimento da Silva.

Tarefa realizada por cada um dos integrantes: Ambos os integrantes do grupo estavam, ao mesmo momento, fazendo as alterações necessárias para completar o exercício, usando apenas um computador e comentando sobre o exercício.

Objetivo: Resolver o problema da senha de forma que você não forneça a informação do tamanho da senha para a função que gera a população. Considere que a senha pode ser uma string de 1 até 30 caracteres.

Resolução: Para resolver esse problema, fizemos alterações na função de gerar um candidato que passa a gerar candidatos com um tamanho aleatório. Para a função objetivo, adicionamos na métrica de distância um valor que corresponde a diferença de tamanho entre o candidato e a senha original. Sendo assim, quanto menor essa diferença, menor será a distância entre ambos. Entretanto, a distância entre caracteres continua válida.

Adaptamos o código da função cruzamento para funcionar se os indivíduos pai e mãe fossem de tamanho diferente. Se o maior indivíduo for o pai, adaptamos ele, deixando-o do mesmo tamanho da mãe, e vice-versa, retirando os n últimos genes do pai. Dessa forma, garantimos que eles possam trocar seus genes. Entretanto, para não perdermos a informação extra do maior indivíduo, nós adicionamos os genes extras do maior indivíduo no filho 2.

No contexto de algoritmos genéticos aplicados à evolução de senhas, é essencial permitir mutações que alterem o tamanho das cadeias de caracteres. Caso contrário, se a população não contiver indivíduos com o mesmo número de caracteres da senha-alvo, o algoritmo pode nunca convergir para a solução correta. Para lidar com isso, implementamos uma mutação específica que modifica o tamanho da senha. Quando essa mutação ocorre, o indivíduo pode ganhar ou perder um gene, com 50% de chance para cada possibilidade. A única exceção é quando o indivíduo possui apenas um gene — nesse caso, não é permitido remover um gene, pois a existência de uma senha vazia não é válida no contexto do problema.

Conclusão e aprendizado: As alterações conseguiram resolver o problema. Algo que percebemos na forma como o algoritmo funciona foi que ele forma uma escada se aproximando da senha correta se olharmos para a senha que corresponde ao maior fit observado. Isso se deve ao fato de que nesse código é mais provável alcançar uma distância maior da senha com um tamanho maior para ela, devido à chance maior de ter caracteres errados.

Onde está a tarefa: Essa Fera está na pasta `Feras_formidaveis/Fera_4.9` no [GitHub](#) com o título `Fera_4.9.ipynb`.

11 Atividade 4.10

Nome: O caixeiro que prefere cidades ímpares

Objetivo: Encontre o caminho de menor distância no problema do caixeiro viajante que prefere cidades ímpares e mostre ele de forma gráfica.

Considerações do experimento: Considere um número n maior ou igual a 7 de coordenadas (x, y) de cidades (cada cidade ocupa uma posição (x, y) diferente). Você pode gerar as coordenadas de forma aleatória ou simplesmente usar as coordenadas que desejar. O caixeiro só anda em linha reta e apenas entre duas cidades. O caixeiro começa e termina seu trajeto na mesma cidade e, fora a cidade inicial, ele não visita nenhuma outra cidade mais de uma vez. Além disso, atribua um número inteiro para cada uma das n cidades que o caixeiro irá visitar, iniciando a contagem pelo número zero e aumentando esse número de 1 em 1. O caixeiro deverá necessariamente visitar primeiro as cidades com números ímpares antes das cidades com números pares. A cidade de número zero deve ser a cidade inicial.

Resolução: Para utilizar algoritmos genéticos nessa situação a minha ideia principal foi fazer com que em nenhum momento o algoritmo trabalhasse com indivíduos inválidos. E para fazer isso, o primeiro passo é criar apenas indivíduos válidos, o que foi feito por meio da função de criar os candidatos, que separa as cidades pelas ímpares e pelas pares (com exceção da cidade 0), após, as cidades são embaralhadas enquanto elas ainda estão separadas em ímpares e pares, por fim, é necessário apenas definir o caminho concatenado uma lista contendo a "Cidade 0" com as cidades ímpares e, por fim, as cidades pares.

No cruzamento também é necessário fazer mudanças, essas mudanças foram feitas implementando uma nova função de cruzamento que separa cada pai e mãe na parte par e na parte ímpar de cada um deles e cruza, pela função de cruzamento ordenado, a parte par do pai com a parte par da mãe, o mesmo ocorre para a parte ímpar. Após isso, é necessário apenas formar o filho, concatenando a lista com a "Cidade 0" com a parte ímpar do pai e com a parte par do pai, o mesmo ocorre para a mãe.

Na mutação a ideia é adaptar a função de mutacao por troca criando uma função que sorteia o `gene1`, se esse `gene1` pertencer à parte ímpar do indivíduo ele gera um `gene2` que também pertence à parte ímpar do indivíduo, e o mesmo ocorre para a parte par.

Conclusão e aprendizado: O problema foi resolvido de forma simples, foi interessante perceber o quão bom é não precisar trabalhar com indivíduos inválidos. De uma maneira geral as únicas mudanças necessárias ocorreram apenas na criação, mutação e cruzamento dos indivíduos, que poderiam mudar o formato dos indivíduos.

Onde está a tarefa: Essa Fera está na pasta `Feras_formidaveis/Fera_4.10` no [GitHub](#) com o título `Fera_4.10.ipynb`.

12 Atividade 4.12

Nome: Novos palíndromos

Objetivo: Encontre pelo menos 10 palíndromos diferentes de 5 letras. Estes palíndromos devem ter pelo menos uma vogal. Não é necessário que eles formem palavras válidas em português ou qualquer outro idioma.

Resolução: A alteração do algoritmo foi realizada na função objetivo. Basicamente foi necessário definir quais são as vogais e criar uma variável booleana chamada que inicialmente começa como False, se houver pelo menos uma vogal no indivíduo essa variável fica definida como True. Essa parte é importante porque se for False a distância é definida como 0.01, penalizando o indivíduo. Se for True um for é percorrido pegando cada letra do candidato e a sua complementar, que é para percorrendo o candidato de trás para frente. Exemplo: se a palavra for nivel o for vai pegar a letra como sendo n e o complementar como sendo l, após, ele vai pegar a letra i e o complementar e, por fim, ambos pegarão a letra v.

Para evitar que o resto da lista fosse percorrido, o que seria inútil, eu defini o centro da minha palavra como sendo a aproximação para mais do len da lista candidato dividido por dois a partir da função `math.ceil`, dessa forma, para palavras com len par ele para realmente em $\text{len}/2$ e para len impar ele para em $\text{len}/2 + 0.5$, garantindo que ambas as letras sejam observadas. Eu dou um break no for se essa letra do centro já tiver sido observada.

Após pegar essas letras eu somo na distância o absoluto no número que representa a letra (pela função `ord()`) pelo número que representa o complementar.

Conclusão e aprendizado: O código funcionou bem, foi simples modificar a estrutura do algoritmo da senha para encontrar palíndromos. Algo perceptível é que na maioria das vezes a vogal aparece no centro.

Onde está a tarefa: Essa Fera está na pasta `Feras_formidaveis/Fera_4.12` no [GitHub](#) com o título `Fera_4.12.ipynb`.

13 Reflexões sobre a jornada

Foi muito bom ter essa matéria, a forma como ela funcionou em termos de organização deixou uma experiência riquíssima permitiu com que eu estivesse apto a me aventurar na aplicação dos conceitos de redes neurais e algoritmos genéticos em meus futuros trabalhos na carreira científica.

Em relação às atividades realizadas presentes nesse memorial destaco quatro com histórias interessantes: a 3.6 que foi a última que fiz porque só fiz ela pois percebi que eu já havia feito uma curva de aprendizado em outras feras; a 4.2 porque a estratégia de Early Stopping é minha favorita; a 4.3 porque foi a mais difícil para mim; e a 4.10 que abri o arquivo python, fiz as alterações necessárias e achei que ele iria rodar certinho, obviamente não rodou certinho.

14 Referências

- [1] Cassar, Daniel R. Material didático de redes neurais e algoritmos genéticos do ano de 2025.
- [2] Tabela Periódica - Ptable. Disponível em: <https://ptable.com/?lang=pt#Propriedades>.
- [3] Copa do Mundo FIFA de 2002 – Wikipédia, a enciclopédia livre. Disponível em: https://pt.wikipedia.org/wiki/Copa_do_Mundo_FIFA_de_2002.
- [4] HUNNER, T. Every dunder method in Python. Disponível em: <https://www.pythonsmells.com/every-dunder-method/>. Acesso em: 1 abr. 2025.
- [5] NO. tocando arquivos mp3 no Python. Disponível em: <https://pt.stackoverflow.com/questions/449082/tocando-arquivos-mp3-no-python>. Acesso em: 1 abr. 2025.
- [6] FELIPE MAX. Brasil x Alemanha Melhores Momentos Final Copa do Mundo 2002. Disponível em: <https://www.youtube.com/watch?v=6RFu-SYt-L8>. Acesso em: 10 jun. 2025.
- [7] CECCON, D. Funções de ativação: definição, características, e quando usar cada uma – IA Expert Academy. Disponível em: <https://iaexpert.academy/2020/05/25/funcoes-de-ativacao-definicao-caracteristicas-e-quando-usar-cada-uma/>.
- [8] Cross-entropy. Disponível em: <https://en.wikipedia.org/wiki/Cross-entropy>.
- [9] What is a state dict in PyTorch — PyTorch Tutorials 2.4.0+cu121 documentation. Disponível em: https://pytorch.org/tutorials/recipes/recipes/what_is_state_dict.html.
- [10] KASHYAP, P. Early Stopping in Deep Learning: A Simple Guide to Prevent Overfitting. Disponível em: <https://medium.com/@piyushkashyap045/early-stopping-in-deep-learning-a-simple-guide-to-prevent-overfitting-1073f56b493e>.
- [11] Papers with Code - Early Stopping Explained. Disponível em: <https://paperswithcode.com/method/early-stopping>.
- [12] DSA, E. Capítulo 23 - Como Funciona o Dropout? - Deep Learning Book. Disponível em: <https://www.deeplearningbook.com.br/como-funciona-o-dropout/>. Acesso em: 27 maio. 2025.
- [13] YADAV, H. Dropout in Neural Networks — Towards Data Science. Disponível em: <https://towardsdatascience.com/dropout-in-neural-networks-47a162d621d9/>.
- [14] KASHYAP, P. Understanding Dropout in Deep Learning: A Guide to Reducing Overfitting. Disponível em: <https://medium.com/@piyushkashyap045/understanding-dropout-in-deep-learning-a-guide-to-reducing-overfitting-26cbb68d5575>.
- [15] INDURAJ. Different Variants of gradient descent. Disponível em: <https://induraj2020.medium.com/different-variants-of-gradient-descent-6854d3118c4c>.
- [16] DSA, E. Capítulo 12 - Aprendizado Com a Descida do Gradiente. Disponível em: <https://www.deeplearningbook.com.br/aprendizado-com-a-descida-do-gradiente/>.

[17] WIKIPEDIA CONTRIBUTORS. Gradient descent. Disponível em: https://en.wikipedia.org/wiki/Gradient_descent.

[18] BHARGAVI POYEKAR. Building simple Neural Networks using Pytorch (NN, CNN) for MNIST dataset. Disponível em: <https://medium.com/%40bpoyeka1/building-simple-neural-networks-nn-cnn-using-pytorch-for-mnist-dataset-31e459d17788>. Acesso em: 2 maio 2025.